

Learn Java HashMap

Yesterday 2:02 PM



Can u teach me HashMap in java with complete details and all its important functions syntax

Compare with other AI ⚡

Absolutely. Let's learn **Java HashMap from zero to interview/DSA level**, including the syntax of the important methods and when to use each one.

The key idea is very simple:

HashMap stores data as key → value pairs.

For example:

You give a **key**, and HashMap gives you its **value**.

1. What is HashMap?

HashMap is a class in Java's `java.util` package.

Basic declaration:

Example:

Here:

- **String** = type of key
- **Integer** = type of value

So we can store:

2. Creating a HashMap

Empty HashMap

With different data types

Example:

Result:

3. The MOST important function: `put()`

`put()` adds a key-value pair.

Syntax

Example:

Now:

4. What happens if the key already exists?

This is **VERY important**.

The second `put()` **updates** the existing value.

So:

There cannot be two separate entries with the same key.

Think:

Instead:

5. `get()`

Used to retrieve a value using its key.

Syntax

Example:

Output:

Another:

Output:

6. What if the key doesn't exist?

Output:

So:

returns:

- value if key exists
- `null` if key doesn't exist

7. `containsKey()`

This is one of the **most important HashMap functions for DSA**.

It checks whether a key exists.

Syntax

Returns:

Example:

You can also do:

8. containsValue()

Checks whether a particular value exists.

Syntax

Example:

Output:

However, in DSA you'll use `containsKey()` **much more frequently**.

9. remove()

Removes an entry using its key.

Syntax

Java

```
map.remove(key);
```

Example:

Java

```
map.put("Apple", 100);  
map.put("Mango", 80);  
  
map.remove("Apple");
```

Now only:

```
Mango → 80
```

remains.

10. size()

Returns the number of key-value pairs.

Syntax

Java

```
map.size();
```

Example:

Java

```
HashMap<String, Integer> map = new HashMap<>();

map.put("Apple", 100);
map.put("Mango", 80);
map.put("Banana", 60);

System.out.println(map.size());
```

Output:

```
3
```

11. isEmpty()

Checks whether HashMap contains anything.

Syntax

Java

```
map.isEmpty();
```

Returns:

```
true / false
```

Example:

Java

```
if (map.isEmpty()) {
    System.out.println("Map is empty");
}
```

12. clear()

Removes **everything** from the HashMap.

Syntax

Java

```
map.clear();
```

Example:

Java

```
map.put("A", 10);
map.put("B", 20);
```

```
map.clear();
```

Now:

```
{}
```

And:

```
Java
System.out.println(map.isEmpty());
```

returns:

```
true
```

13. Getting all keys: keySet()

This is extremely important for traversing a HashMap.

Syntax

```
Java
map.keySet();
```

Example:

```
Java
HashMap<String, Integer> map = new HashMap<>();

map.put("Apple", 100);
map.put("Mango", 80);
map.put("Banana", 60);

for (String key : map.keySet()) {
    System.out.println(key);
}
```

Possible output:

```
Apple
Mango
Banana
```

⚠ Important: HashMap does NOT guarantee this order.

You might get:

```
Banana
Apple
Mango
```

or another order.

14. Getting all values: values()

Syntax

```
Java
map.values();
```

Example:

```
Java
for (Integer value : map.values()) {
    System.out.println(value);
}
```

Possible output:

```
100
80
60
```

Again, order is not guaranteed.

15. Getting both key AND value: entrySet()

This is another **VERY important DSA concept**.

Syntax

```
Java
map.entrySet();
```

Example:

```
Java
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " " + entry.getValue());
}
```

Output could be:

```
Apple 100
Mango 80
Banana 60
```

You need:

```
Java
import java.util.Map;
```

or simply:

```
Java
import java.util.*;
```

16. getKey()

Used with an entry.

```
Java
entry.getKey();
```

Example:

```
Java
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey());
}
```

17. getValue()

Gets the value of an entry.

```
Java
entry.getValue();
```

Example:

```
Java
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getValue());
}
```

18. The 3 ways to traverse HashMap

You should know all three.

Method 1: keys

```
Java
for (String key : map.keySet()) {
    System.out.println(key);
}
```

Method 2: values

```
Java
for (Integer value : map.values()) {
    System.out.println(value);
}
```

Method 3: key + value

Java

```
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " " + entry.getValue());
}
```

For DSA, `entrySet()` and `keySet()` are particularly useful.

19. `getOrDefault()`

 **VERY IMPORTANT for DSA.**

This method is used constantly for frequency counting.

Syntax

Java

```
map.getOrDefault(key, defaultValue);
```

Suppose:

Java

```
HashMap<Integer, Integer> map = new HashMap<>();
```

We want to count frequencies.

Without `getOrDefault()`:

Java

```
if (map.containsKey(x)) {
    map.put(x, map.get(x) + 1);
} else {
    map.put(x, 1);
}
```

That's a little annoying.

With `getOrDefault()`:

Java

```
map.put(x, map.getOrDefault(x, 0) + 1);
```

Beautiful little DSA weapon. 

20. Frequency counting example

Suppose:

Java

```
int[] arr = {1, 2, 2, 3, 1, 2};
```

We want:


```
1 → 2
2 → 3
3 → 1
```

Code:

```
Java
HashMap<Integer, Integer> map = new HashMap<>();

for (int x : arr) {
    map.put(x, map.getOrDefault(x, 0) + 1);
}
```

Let's understand:

For first 1:

```
Java
map.getOrDefault(1, 0)
```

returns 0.

Then:

```
Java
0 + 1 = 1
```

So:

```
1 → 1
```

Next 2:

```
2 → 1
```

Next 2:

```
2 → 2
```

And so on.

Final:

```
1 → 2
2 → 3
3 → 1
```

🔥 Memorize this pattern:

```
Java
map.put(x, map.getOrDefault(x, 0) + 1);
```

You will see it everywhere in DSA.

21. putIfAbsent()

Adds a value **only if the key doesn't already exist**.

Syntax

```
Java
map.putIfAbsent(key, value);
```

Example:

```
Java
map.put("Apple", 100);
map.putIfAbsent("Apple", 200);
```

The value remains:

```
Apple → 100
```

Because **Apple** already existed.

But:

```
Java
map.putIfAbsent("Mango", 80);
```

adds:

```
Mango → 80
```

Difference

```
Java
put()
```

can overwrite.

```
Java
putIfAbsent()
```

doesn't overwrite an existing mapping.

22. replace()

Changes the value associated with a key.

Syntax

Java

```
map.replace(key, newValue);
```

Example:

Java

```
map.put("Apple", 100);  
map.replace("Apple", 200);
```

Now:

```
Apple → 200
```

23. replace() with old value

There is another version:

Java

```
map.replace(key, oldValue, newValue);
```

Example:

Java

```
map.put("Apple", 100);  
map.replace("Apple", 100, 200);
```

Since the current value is **100**, it becomes:

```
Apple → 200
```

But:

Java

```
map.replace("Apple", 500, 300);
```

doesn't work because the current value isn't **500**.

24. replaceAll()

Can modify all values.

Syntax

Java

```
map.replaceAll((key, value) -> newValue);
```

Example:

Java

```
HashMap<String, Integer> map = new HashMap<>();

map.put("A", 10);
map.put("B", 20);
map.put("C", 30);

map.replaceAll((key, value) -> value * 2);
```

Now:

```
A → 20
B → 40
C → 60
```

This is useful, but **not nearly as important as put, get, containsKey, and getOrDefault for beginner DSA.**

25. remove() with key and value

You can also remove an entry only if both match.

Syntax

Java

```
map.remove(key, value);
```

Example:

Java

```
map.put("Apple", 100);

map.remove("Apple", 100);
```

It removes it.

But:

Java

```
map.remove("Apple", 500);
```

doesn't remove it because the value doesn't match.

26. getOrDefault() VS get()

Very important distinction.

get()

Java

```
map.get("Apple");
```

If key doesn't exist:

```
null
```

getOrDefault()

Java

```
map.getOrDefault("Apple", 0);
```

If key doesn't exist:

```
0
```

That's why it is so useful for counting.

27. Can HashMap have duplicate keys?

No.

Java

```
map.put(1, "A");
map.put(1, "B");
```

Final:

```
1 → B
```

The second value replaces the first.

28. Can HashMap have duplicate values?

Yes.

Java

```
map.put(1, "A");
map.put(2, "A");
map.put(3, "A");
```

Perfectly valid:

```
1 → A
2 → A
3 → A
```

Keys must be unique.

Values don't have to be.

29. Can HashMap contain null?

Yes.

A normal Java **HashMap** allows:

- one **null** key
- multiple **null** values

Example:

Java

```
HashMap<String, Integer> map = new HashMap<>();

map.put(null, 100);
map.put("Apple", null);
map.put("Mango", null);
```

Valid.

30. Does HashMap maintain insertion order?

No.

If you do:

Java

```
map.put("A", 1);
map.put("B", 2);
map.put("C", 3);
```

You cannot assume iteration gives:

A
B
C

If you specifically need insertion order, use:

Java

LinkedHashMap

If you need sorted keys, use:

Java

TreeMap

We'll come back to this comparison.

31. HashMap vs LinkedHashMap vs TreeMap

Feature	HashMap	LinkedHashMap	TreeMap
Key-value	✓	✓	✓

Feature	HashMap	LinkedHashMap	TreeMap
Insertion order	✗	✓	✗
Sorted keys	✗	✗	✓
Average get()	O(1)	O(1)	O(log n)
Average put()	O(1)	O(1)	O(log n)
Allows null key	✓	✓	Usually ✗
DSA frequency counting	★★★★	★	★

For most basic DSA problems:

Java
HashMap

is your default choice.

32. HashMap time complexity

This matters a lot for DSA.

Operation	Average
put()	O(1)
get()	O(1)
containsKey()	O(1)
remove()	O(1)
containsValue()	O(n)
size()	O(1)
isEmpty()	O(1)
clear()	O(n)

The reason HashMap is so powerful is that:

```
get → O(1)
put → O(1)
search by key → O(1)
```

on average.

33. Why is HashMap so fast?

Under the hood, HashMap uses **hashing**.

Imagine:

```
Key
↓
hash function
```

```

↓
hash value
↓
bucket
↓
stored value

```

For example:

```

"Apple"
↓
hashCode()
↓
some integer
↓
bucket
↓
100

```

When you later ask:

```

Java
map.get("Apple");

```

Java calculates the hash again and quickly locates the corresponding bucket.

That's the magic behind approximately $O(1)$ average lookup.

34. hashCode() and equals()

These two concepts are important when you eventually learn HashMap internals.

HashMap relies on:

```

Java
hashCode()

```

and

```

Java
equals()

```

to determine where a key belongs and whether two keys should be considered equal.

For your current DSA level, remember:

HashMap uses hashing to efficiently locate keys.

Later, when you study Java Collections deeply, learn collisions, buckets, load factor, resizing, and treeification.

35. The most important DSA pattern: frequency map

Suppose:

Java

```
int[] arr = {1, 1, 2, 2, 2, 3};
```

Count frequencies:

Java

```
HashMap<Integer, Integer> freq = new HashMap<>();  
  
for (int x : arr) {  
    freq.put(x, freq.getOrDefault(x, 0) + 1);  
}
```

Then:

```
1 → 2  
2 → 3  
3 → 1
```

36. Finding frequency of a particular number

Java

```
int frequency = freq.getOrDefault(2, 0);
```

Result:

```
3
```

If you ask:

Java

```
int frequency = freq.getOrDefault(10, 0);
```

Result:

```
0
```

37. Finding duplicates

Example:

Java

```
int[] arr = {1, 2, 3, 2, 4, 1};
```

Code:

Java

```
HashMap<Integer, Integer> map = new HashMap<>();  
  
for (int x : arr) {  
    map.put(x, map.getOrDefault(x, 0) + 1);  
}  
  
for (int key : map.keySet()) {
```

```

    if (map.get(key) > 1) {
        System.out.println(key);
    }
}

```

Output:

```

1
2

```

38. HashMap with String

Very common in DSA.

```

Java
HashMap<String, Integer> map = new HashMap<>();

map.put("apple", 1);
map.put("banana", 2);
map.put("orange", 3);

```

Retrieve:

```

Java
System.out.println(map.get("banana"));

```

Output:

```

2

```

39. Character frequency

Extremely common interview problem.

```

Java
String s = "banana";

HashMap<Character, Integer> map = new HashMap<>();

for (char ch : s.toCharArray()) {
    map.put(ch, map.getOrDefault(ch, 0) + 1);
}

```

Result:

```

b → 1
a → 3
n → 2

```

This pattern is worth memorizing.

40. First non-repeating character pattern

HashMap can help you solve problems like:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

```
"leetcode"
```

Find the first character whose frequency is 1.

First:

```
Java
HashMap<Character, Integer> freq = new HashMap<>();

for (char ch : s.toCharArray()) {
    freq.put(ch, freq.getOrDefault(ch, 0) + 1);
}
```

Then:

```
Java
for (char ch : s.toCharArray()) {
    if (freq.get(ch) == 1) {
        System.out.println(ch);
        break;
    }
}
```

The key idea:

```
First pass → count
Second pass → find
```

This two-pass HashMap pattern appears constantly in DSA.

41. HashMap with Integer keys

```
Java
HashMap<Integer, Integer> map = new HashMap<>();
```

Example:

```
Java
map.put(10, 100);
map.put(20, 200);
```

Retrieve:

```
Java
int x = map.get(10);
```

42. HashMap with Character keys

```
Java
HashMap<Character, Integer> map = new HashMap<>();
```

Example:

Java

```
map.put('a', 10);
map.put('b', 20);
```

43. HashMap with String keys and String values

Java

```
HashMap<String, String> map = new HashMap<>();
```

Example:

Java

```
map.put("India", "Delhi");
map.put("France", "Paris");
```

44. HashMap with lists as values

This is a more advanced and **very useful DSA pattern**.

Java

```
HashMap<Integer, ArrayList<Integer>> map = new HashMap<>();
```

For example:

```
1 → [10, 20, 30]
2 → [40, 50]
```

You can create it like:

Java

```
HashMap<Integer, ArrayList<Integer>> map = new HashMap<>();

map.put(1, new ArrayList<>());
map.get(1).add(10);
map.get(1).add(20);
```

Now:

```
1 → [10, 20]
```

45. computeIfAbsent()

This is useful for the previous pattern.

Instead of:

Java

```
if (!map.containsKey(1)) {
    map.put(1, new ArrayList<>());
}

map.get(1).add(10);
```

You can write:

Java

```
map.computeIfAbsent(1, k -> new ArrayList<>()).add(10);
```

This becomes especially useful in:

- Graphs
- Grouping
- Adjacency lists
- Mapping one key to multiple values

You don't need to memorize this on day one, but **recognize it**.

46. forEach()

You can also iterate using:

Java

```
map.forEach((key, value) -> {
    System.out.println(key + " " + value);
});
```

Example:

Java

```
map.forEach((key, value) ->
    System.out.println(key + " → " + value)
);
```

This is clean Java syntax, although traditional `entrySet()` is perfectly fine for DSA.

47. Complete HashMap cheat sheet

Here is the part I'd recommend you actually memorize:

Java

```
import java.util.HashMap;
import java.util.Map;
```

Create

Java

```
HashMap<Integer, Integer> map = new HashMap<>();
```

Add / update

Java

```
map.put(key, value);
```

Get

```
Java  
map.get(key);
```

Check key

```
Java  
map.containsKey(key);
```

Check value

```
Java  
map.containsValue(value);
```

Remove

```
Java  
map.remove(key);
```

Number of entries

```
Java  
map.size();
```

Empty?

```
Java  
map.isEmpty();
```

Delete everything

```
Java  
map.clear();
```

Get default

```
Java  
map.getOrDefault(key, defaultValue);
```

Add only if absent

```
Java  
map.putIfAbsent(key, value);
```

Replace value

```
Java
map.replace(key, newValue);
```

Get all keys

```
Java
map.keySet();
```

Get all values

```
Java
map.values();
```

Get key-value pairs

```
Java
map.entrySet();
```

Get key from entry

```
Java
entry.getKey();
```

Get value from entry

```
Java
entry.getValue();
```

48. The 5 functions you should master FIRST

Don't try to memorize 20 methods at once. For **DSA**, master these first:

① put()

```
Java
map.put(key, value);
```

② get()

```
Java
map.get(key);
```

③ containsKey()

```
Java
```

```
map.containsKey(key);
```

④ `getOrDefault()`

Java

```
map.getOrDefault(key, 0);
```

⑤ `keySet()`

Java

```
for (int key : map.keySet()) {
    // ...
}
```

Then learn:

Java

```
entrySet()
remove()
size()
putIfAbsent()
```

49. One complete example

Let's put everything together.

Java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        HashMap<String, Integer> map = new HashMap<>();

        // Add
        map.put("Apple", 100);
        map.put("Mango", 80);
        map.put("Banana", 60);

        // Get
        System.out.println(map.get("Apple"));

        // Check key
        System.out.println(map.containsKey("Mango"));

        // Check value
        System.out.println(map.containsValue(60));

        // Size
        System.out.println(map.size());

        // Remove
        map.remove("Banana");

        // Add only if absent
        map.putIfAbsent("Orange", 70);

        // Get with default
        System.out.println(map.getOrDefault("Grapes", 0));

        // Traverse keys
        for (String key : map.keySet()) {
```



```
System.out.println(key);
```

50. One mental model to remember everything

Think of a HashMap as a **dictionary** 📖.

If you have:

Word	Meaning
-----	-----
apple	fruit
java	programming language
cat	animal

Then:

```
Java
map.get("java")
```

means:

"Dictionary, give me the meaning associated with **java**."

And:

```
Java
map.put("python", "programming language");
```

means:

"Dictionary, add this new word."

And:

```
Java
map.containsKey("java")
```

means:

"Dictionary, do you have the word **java**?"

And:

```
Java
map.getDefault("ruby", "not found")
```

means:

"Give me Ruby's meaning, but if Ruby isn't there, give me **not found**."

That mental model makes HashMap much less intimidating.

★ Your DSA HashMap toolkit

When solving problems, these are the patterns I want you to recognize immediately:

Java

```
HashMap<Integer, Integer> map = new HashMap<>();
```

Frequency:

Java

```
map.put(x, map.getOrDefault(x, 0) + 1);
```

Check existence:

Java

```
if (map.containsKey(x)) {
    // exists
}
```

Get frequency:

Java

```
map.getOrDefault(x, 0);
```

Traverse:

Java

```
for (int key : map.keySet()) {
    // ...
}
```

Key + value:

Java

```
for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
    int key = entry.getKey();
    int value = entry.getValue();
}
```

If you're preparing for **Striver/placement DSA**, HashMap is especially important for problems involving **frequency counting, duplicates, Two Sum, first unique element, anagrams, subarrays, prefix sums, and counting pairs**. Once these patterns click, a surprisingly large chunk of array/string problems starts looking much less scary. 🧠⚡

ChatGPT can make mistakes. Check important info.